```python
import time
import os
import cv2
import csv
import threading
import numpy as np
import serial
from datetime import datetime
from pyzbar.pyzbar import decode

import board
import busio
from adafruit_bus_device.i2c_device import I2CDevice
import adafruit_vl53l0x


##################################################
# CONFIG
##################################################

USB_CAMERA_INDEX = 0   # caméra USB = /dev/video0
CHANNEL_LIGNE_GAUCHE = 0
CHANNEL_LIGNE_DROITE = 1
CHANNEL_VERTICAUX = [3, 5, 6, 7]

SEUIL_PASSAGE = 200
DELAI_LOOP = 0.001

XBEE_PORT = "/dev/ttyUSB0"
XBEE_BAUD = 9600


##################################################
# XBEE — ENVOI FIABLE
##################################################

def init_xbee():
    try:
        print("[XBEE] Initialisation...")
        x = serial.Serial(XBEE_PORT, XBEE_BAUD, timeout=1, write_timeout=1)
        time.sleep(1)
        print("[XBEE] OK")
        return x
    except Exception as e:
        print(f"[XBEE][ERREUR] Impossible d'ouvrir {XBEE_PORT} : {e}")
        return None

xbee = init_xbee()
```

```
def xbee_send_line(msg, repeat=5):
    if xbee is None or not xbee.is_open:
        print(f"[XBEE][WARN] '{msg}' non envoyé (port fermé)")
        return
    data = (msg + "\r\n").encode()
    for i in range(repeat):
        try:
            xbee.write(data)
            xbee.flush()
            print(f"[XBEE] → {msg} ({i+1}/{repeat})")
            time.sleep(0.08)
        except Exception as e:
            print(f"[XBEE][ERREUR] {e}")
            break


################################################
# CAMERA USB — THREAD + RECONNEXION
################################################

last_frame = None
camera_running = True

def camera_thread():
    global last_frame, camera_running

    while camera_running:
        cap = cv2.VideoCapture(USB_CAMERA_INDEX, cv2.CAP_V4L2)

        if not cap.isOpened():
            print("[CAM] Caméra USB non détectée, retry...")
            time.sleep(1)
            continue

        print("[CAM] Caméra USB OK")

        cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
        cap.set(cv2.CAP_PROP_FPS, 30)

        while camera_running:
            ret, frame = cap.read()
            if ret:
                last_frame = frame
            else:
                print("[CAM] Perte du flux USB, reconnexion...")
                break
```

```python
        cap.release()

threading.Thread(target=camera_thread, daemon=True).start()
time.sleep(1)


################################################
# I2C + MULTIPLEXEUR
################################################

i2c = busio.I2C(board.SCL, board.SDA, frequency=400000)
tca = I2CDevice(i2c, 0x70)

def select_channel(ch):
    with tca as d:
        d.write(bytes([1 << ch]))
    time.sleep(0.002)

def init_capteur(ch, retries=3):
    for attempt in range(retries):
        try:
            select_channel(ch)
            cap = adafruit_vl53l0x.VL53L0X(i2c)
            print(f"[I2C] Capteur OK sur canal {ch}")
            return cap
        except Exception:
            time.sleep(0.05)
    print(f"[I2C][WARN] Capteur absent sur canal {ch}")
    return None


################################################
# INITIALISATION DES CAPTEURS
################################################

print("Initialisation des capteurs...")

capteur_ligne_gauche = init_capteur(CHANNEL_LIGNE_GAUCHE)
capteur_ligne_droite = init_capteur(CHANNEL_LIGNE_DROITE)

capteurs_verticaux = []
canaux_verticaux_actifs = []

for ch in CHANNEL_VERTICAUX:
    c = init_capteur(ch)
    if c:
        capteurs_verticaux.append(c)
```

```
        canaux_verticaux_actifs.append(ch)

print(f"[I2C] Capteurs verticaux actifs : {canaux_verticaux_actifs}")


################################################
# QR CODE — VERSION OPTIMISÉE
################################################

def lire_qr_robuste(image):
    if image is None:
        return None

    h, w = image.shape[:2]
    crop = image[int(h*0.2):int(h*0.8), int(w*0.2):int(w*0.8)]

    gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)

    codes = decode(gray)
    if codes:
        return codes[0].data.decode()

    _, th = cv2.threshold(gray, 0, 255, cv2.THRESH_OTSU)
    codes = decode(th)
    if codes:
        return codes[0].data.decode()

    bl = cv2.GaussianBlur(gray, (5, 5), 0)
    codes = decode(bl)
    if codes:
        return codes[0].data.decode()

    return None


################################################
# ARCHIVAGE
################################################

BASE_DIR = "/home/pi/capture_covaciel"
date_course = datetime.now().strftime("%d-%m-%Y_%H-%M-%S")
course_dir = os.path.join(BASE_DIR, date_course)
os.makedirs(course_dir, exist_ok=True)

csv_path = os.path.join(course_dir, "resultats.csv")

with open(csv_path, "w", newline="") as f:
    writer = csv.writer(f)
```

```
    writer.writerow(["voiture_id", "tour", "temps_tour", "temps_total", "timestamp", "photo"])


###############################################
# NB TOURS
###############################################

def demander_nb_tours():
    while True:
        try:
            n = int(input("Combien de tours ? "))
            if n > 0:
                return n
        except:
            pass
        print("Nombre invalide.")

nb_tours = demander_nb_tours()


###############################################
# START
###############################################

print("[XBEE] Envoi START...")
xbee_send_line("START")


###############################################
# DETECTION DU SENS
###############################################

def detecter_sens():
    if not capteur_ligne_gauche or not capteur_ligne_droite:
        print("[SENS] Capteurs ligne absents → sens INCONNU")
        return "INCONNU"

    print("[SENS] Détection...")
    while True:
        select_channel(CHANNEL_LIGNE_GAUCHE)
        g = capteur_ligne_gauche.range

        select_channel(CHANNEL_LIGNE_DROITE)
        d = capteur_ligne_droite.range

        if g < SEUIL_PASSAGE and d >= SEUIL_PASSAGE:
            print("→ Sens GAUCHE")
```

```
            return "GAUCHE"
        if d < SEUIL_PASSAGE and g >= SEUIL_PASSAGE:
            print("→ Sens DROITE")
            return "DROITE"

        time.sleep(0.01)

sens = detecter_sens()


################################################
# DETECTION MULTI-VOITURES
################################################

def detecter_voitures_vertical():
    detections = []
    for idx, cap in enumerate(capteurs_verticaux):
        ch = canaux_verticaux_actifs[idx]
        select_channel(ch)
        try:
            if cap.range < SEUIL_PASSAGE:
                detections.append(idx)
        except:
            pass
    return detections


################################################
# LOGIQUE DE COURSE
################################################

depart = {}
tours = {}
temps_total = {}
voitures_finies = set()

print("=== CHRONO MULTI ===")
print(f"Sens détecté : {sens}")
print("En attente de voitures...")


################################################
# BOUCLE PRINCIPALE
################################################

try:
    while True:
        idxs = detecter_voitures_vertical()
```

```
if not idxs:
    time.sleep(DELAI_LOOP)
    continue

for idx in idxs:
    ch = canaux_verticaux_actifs[idx]
    print(f"[PASSAGE] Capteur vertical canal {ch}")

    img = last_frame
    if img is None:
        print("[CAM] Pas d'image")
        continue

    voiture_id = lire_qr_robuste(img)
    if voiture_id is None:
        print("[QR] Non détecté")
        continue

    print(f"[QR] Voiture : {voiture_id}")

    if voiture_id not in depart:
        depart[voiture_id] = time.time()
        tours[voiture_id] = 0
        temps_total[voiture_id] = 0.0
        print(f"[DEPART] {voiture_id}")
    else:
        t_tour = time.time() - depart[voiture_id]
        depart[voiture_id] = time.time()
        tours[voiture_id] += 1
        temps_total[voiture_id] += t_tour

        photo = f"voiture_{voiture_id}_tour_{tours[voiture_id]}.jpg"
        cv2.imwrite(os.path.join(course_dir, photo), img)

        with open(csv_path, "a", newline="") as f:
            writer = csv.writer(f)
            writer.writerow([
                voiture_id,
                tours[voiture_id],
                round(t_tour, 3),
                round(temps_total[voiture_id], 3),
                datetime.now().strftime("%H:%M:%S.%f")[:-3],
                photo
            ])

        print(f"[TOUR] {voiture_id} → {t_tour:.3f}s")
```

```
            if tours[voiture_id] >= nb_tours:
                voitures_finies.add(voiture_id)
                print(f"[FIN] {voiture_id} a terminé")

        # -------------------------------
        # FIN DE COURSE → CLASSEMENT FINAL
        # -------------------------------
        if len(voitures_finies) == len(tours) and len(tours) > 0:
            print("\n===== COURSE TERMINÉE =====")

            gagnant = min(voitures_finies, key=lambda v: temps_total[v])
            temps_gagnant = temps_total[gagnant]

            print(f"GAGNANT : Voiture {gagnant} avec {temps_gagnant:.3f} s")

            print("\n--- CLASSEMENT FINAL ---")
            classement = sorted(temps_total.items(), key=lambda x: x[1])

            for pos, (vid, ttot) in enumerate(classement, start=1):
                diff = ttot - temps_gagnant
                print(f"{pos}. Voiture {vid} → {ttot:.3f} s → +{diff:.3f} s")

            print("---------------------------\n")

            print("[XBEE] Envoi END...")
            xbee_send_line("END")
            print("[XBEE] Signal END envoyé.")

            break

        time.sleep(DELAI_LOOP)

except KeyboardInterrupt:
    print("Arrêt manuel.")

camera_running = False
print("Fin du script.")
```